Software Architecture · 课程项目

# 开 题 报 告

Confirmation Report

## 移动云计算环境下的计算划分系统

—— 以 HyperLPR 车牌识别流水线为研究对象

| 成员 | 角色 | 职责分工 |
| --- | --- | --- |
| 饶晓杰 | 组长 | 架构总师；Part C 运行时动态划分决策；答辩主讲 |
| 丁奕轩 | 队员 2 | 端侧系统改造：源码解耦、端侧基线、可插拔流水线与静态划分联调 |
| 陈冠江 | 队员 3 | 云端 FastAPI 多切分推理微服务与网络扰动 |
| 柯　涵 | 队员 4 | 全矩阵基准评测与数据可视化 |

课题类型　计算划分（Computation Partitioning）系统设计与评测
研究对象　HyperLPR 开源中文车牌识别框架（四阶段推理流水线）
提交日期　2026 年 11 月 3 日
线下汇报　2026 年 11 月 4 日　B7-138

本报告中的实测数据均由本团队在统一环境下测得

# 目　录

1. 摘要 ..... 3
2. 引言 ..... 3
    2.1. 研究背景 ..... 3
    2.2. 研究意义 ..... 3
    2.3. 本报告的范围 ..... 3
3. 选定的应用程序：HyperLPR ..... 3
4. 应用程序源码模块结构 ..... 4
    4.1. 模块到文件的分配结构 ..... 4
    4.2. 从“固化流水线”到“可插拔流水线” ..... 5
5. 数据流建模与切分点分析 ..... 6
6. 后续工作计划与里程碑 ..... 7
7. 结论 ..... 7
8. 参考文献 ..... 8

# 1. 摘要

本课题面向移动云计算（Mobile Cloud Computing，MCC）环境，以开源中文车牌识别框架 HyperLPR 为载体，研究**计算划分**（Computation Partitioning）问题：把应用的各计算模块在移动终端与云端之间分配，并通过系统实验刻画“划分方式—网络条件—端到端性能”之间的关系。本报告完成开题阶段的两项工作：其一，对 HyperLPR 源码做模块级剖析，给出模块到文件的分配结构与运行时调用结构，并定位出四阶段推理流水线（目标定位 → 几何对齐 → 字符识别 → 属性分类）中可插拔的接缝——原库把四阶段写死在单一方法内，不改造便无法表达“检测在端侧、识别在云端”这类分工；四算子的解耦实现按里程碑列入中期；其二，对流水线做数据流建模，量化各节点的中间张量体积——确认对齐之后载荷由原图的 385 KB~2 MB 降至 5.81~12.73 KB（对 2 MB 级原图削减 160 倍），据此提出“对齐之后”这一候选切分点。划分方案的设计与多划分方式的性能对比，将按课程里程碑在中期报告（2026-11-29）中给出。

# 2. 引言

## 2.1. 研究背景

移动云计算中的**计算划分**，指把应用软件分解为一组模块，并决定哪些模块在本地执行、哪些部分卸载（offload）到远端服务器或云端。划分点选在哪里，直接决定三件事：

1. **端到端时延** = 端侧计算 + 上行传输 + 云侧计算 + 下行传输；
2. **网络流量**（流量成本与弱网可用性）；
3. **隐私暴露面**（上传完整原图 vs 仅上传局部切片）。

静态划分（离线固定一个切分点）在网络稳定时有效，但真实移动网络会在 5G / WiFi / 4G / 弱网之间反复跳变。同一静态切分点在好网与坏网下的表现可以相差一个数量级，因此课程 Part C 进一步要求实现**运行时动态自适应划分**。而动态决策的前提，是先把应用拆成可独立调度的算子，并拿到每个算子的真实耗时与数据载荷（Cost Profiling）——本报告完成其中的结构基础（接缝界定与数据流建模），算子的解耦实现与完整测量将随划分实验在中期给出。

## 2.2. 研究意义

车牌识别是典型的**计算密集且时延敏感**（compute-intensive & latency-sensitive）的移动视觉应用，广泛出现在智慧停车、辅助驾驶、移动执法等场景。以它为载体的划分研究具有双重意义：方法上，它是一条阶段清晰、中间张量存在数量级体积落差的流水线，划分收益显著且易于量化；应用上，“端侧做轻量预处理 + 云端做重推理”的模式可直接迁移到人脸识别、手势识别、OCR 等同类移动视觉任务。

## 2.3. 本报告的范围

按课程要求，本开题报告展示课题的研究背景及意义（第 1 章）、选定的应用程序（第 2 章）及其源码模块结构（第 3、4 章）。各计算模块在端云之间如何划分、不同划分方式的性能对比，属于中期报告的交付内容；本报告仅在数据流分析中给出切分点的候选位置（第 4 章），并在后续工作计划中列出推进路径（第 5 章）。

# 3. 选定的应用程序：HyperLPR

**定位**：开源高性能中文车牌识别框架，支持单行蓝牌 / 黄牌、新能源绿牌、教练车、双层黄牌等主流车牌类型。

**技术栈**：Python 侧基于 ONNXRuntime（推理引擎）+ OpenCV（图像处理）；Android 端侧基于 MNN 引擎，同一套模型双后端部署。本课题的“端侧”指逻辑上部署于移动终端（Android 设备）的计算角色，绝对时延以真机实测为准。

**选型理由**：

- 天然是一条**四阶段顺序流水线**（定位 → 对齐 → 识别 → 分类），切分点语义清晰；
- 中间张量存在**数量级的体积落差**（原图 MB 级 → 车牌切片 KB 级），划分收益显著且易于量化；
- **纯 CPU 可跑**（Android 真机 1080p 单次识别约 150 ms、内存 84 MB），不需要 GPU 或专用加速器，端侧与云侧可在同类机器上等价复现，实验变量可控；
- 跨平台（Linux / macOS / Android / iOS），代码规模适中（核心包 1697 行），适合做源码级手术。

# 4. 应用程序源码模块结构

本章是开题报告的核心交付。我们从**模块到文件的分配结构**（Module-to-File Allocation）与**运行时调用结构**两个视角剖析源码。

## 4.1. 模块到文件的分配结构

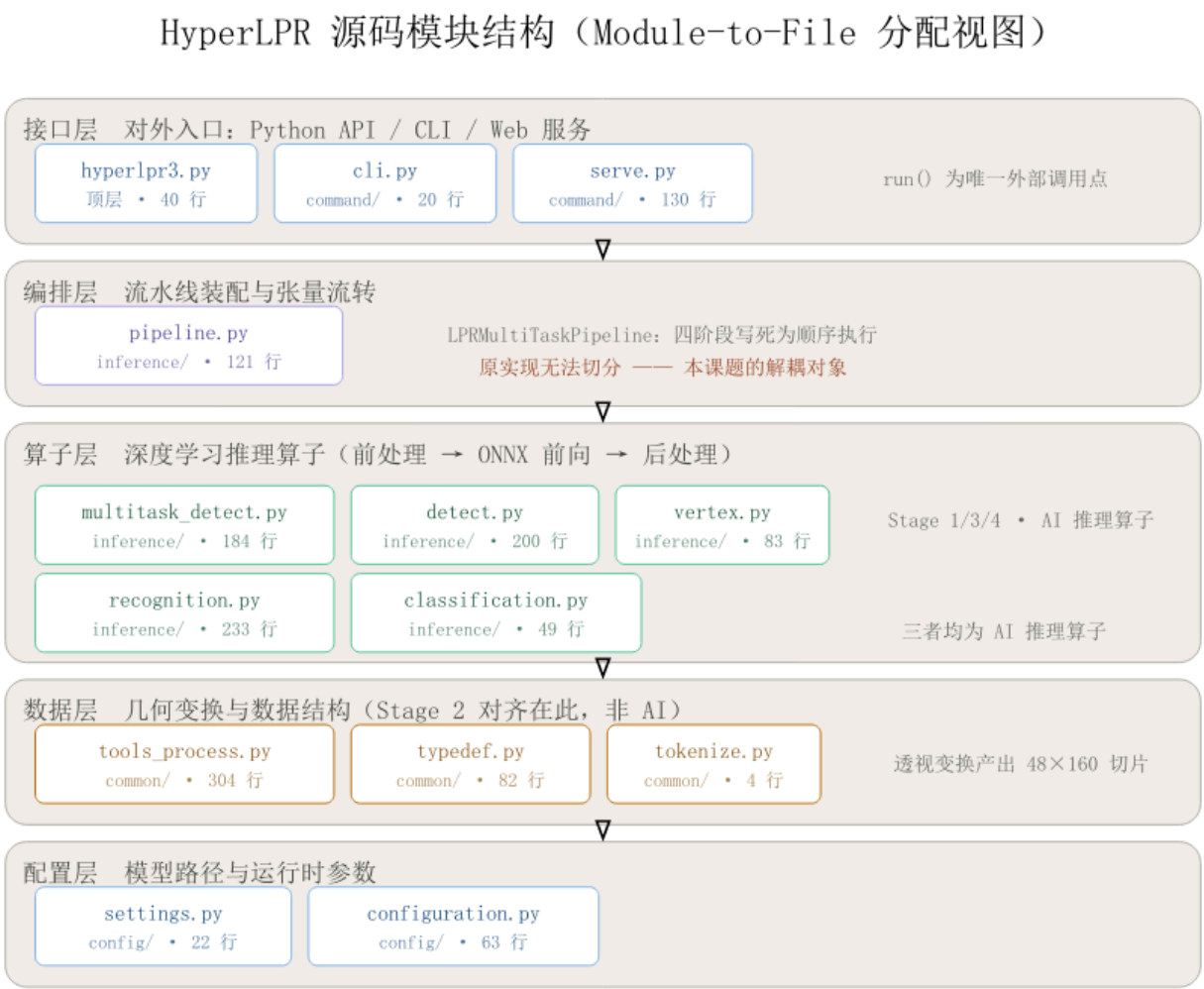



图 1　HyperLPR 源码模块结构（原库 1697 行源码的分层分解：层 → 模块 → 文件 → 行数）

原库可分为五个层次，模块与源文件的对应关系如下表（框内数字为源文件行数）：

| 层次 | 职责 | 关键源文件 | 行数 |
|---|---|---|---|
| 接口层 | 对外暴露识别入口与 CLI / Web 服务 | `hyperlpr3.py`、`command/cli.py`、`command/serve.py` | 190 |
| 编排层 | 把四个算子串成顺序流水线，负责张量与元数据流转 | `inference/pipeline.py`（`LPRMultiTaskPipeline`） | 121 |
| 算子层 | 检测 / 识别 / 分类的 ONNX 会话封装（前处理 → 推理 → 后处理） | `multitask_detect.py`、`detect.py`、`vertex.py`、`recognition.py`、`classification.py` | 749 |
| 数据层 | 图像几何变换（对齐）、类型定义、字符词表 | `common/tools_process.py`、`common/typedef.py` | 386 |
| 配置层 | 模型路径、运行时参数与初始化 | `config/settings.py`、`config/configuration.py` | 85 |

值得注意的是，**四角点由检测阶段一并预测**（YOLOv5 风格多任务头，每锚点输出 15 维），因此“检测 + 对齐”天然可以打包留在端侧，无需为对齐单独跑一个模型；而 Stage 4 分类并非每帧必跑——只有当识别出的车牌号长度 ≥ 7 且规则判不出类型时才触发，属于库内既有的算力优化。

## 4.2. 从“固化流水线”到“可插拔流水线”

原库把四个阶段写死在 `LPRMultiTaskPipeline.run()` 一个方法内：它要么全本地执行、要么整体外移，内部没有“能插拔”的接缝，无法表达“检测在端侧、识别在云端”这类分工。因此后续的关键手术是把 `run()` 拆成四个可单独调用的算子（由外部模块调用原库的推理类，**不修改**原库任何一行），使划分决策从“代码结构”上升为“运行时配置”。本阶段已界定接缝的位置与各算子的输入输出边界，解耦实现按里程碑列入中期（第 5 章计划第 1 项）。

| 阶段 | 实现类 / 函数 | 输入 → 输出 | 算法性质 |
|---|---|---|---|
| Stage 1 定位 | `MultiTaskDetectorORT` | 原图 → 每车 14 维（框 4 + 置信 1 + 四角点 8 + 层标志 1） | 深度学习（YOLOv5 风格多任务头） |
| Stage 2 对齐 | `get_rotate_crop_image` | 原图 + 四角点 → 切片 48×160×3 | 传统 CV（透视变换，纯几何） |
| Stage 3 识别 | `PPRCNNRecognitionORT` | 切片 → 车牌号字符串 + 置信度 | 深度学习（CRNN + CTC 解码） |
| Stage 4 分类 | `ClassificationORT` | 切片 → 类别索引（蓝 / 黄 / 绿） | 深度学习（3 类 CNN，条件触发） |

## 5. 数据流建模与切分点分析

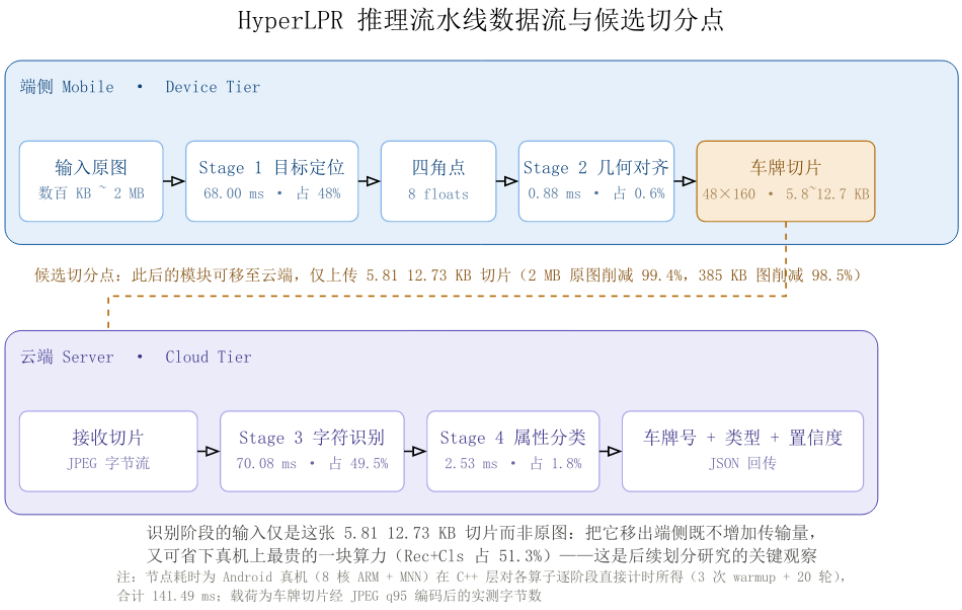



图 2　推理流水线数据流与候选切分点（节点标注为各算子耗时与中间载荷）

各节点张量形态与载荷的定量对比：

| 节点 | 张量形态 | 载荷 | 说明 |
|---|---|---|---|
| 输入原图 | H×W×3，uint8 BGR | 385 KB~2 MB | 1080p 典型 1920×1080 |
| 检测输出 | 1×25200×15，float32 | — | 经置信度过滤 + NMS 后每车剩 14 维 |
| 四角点 | 4×2，int | — | 检测阶段“白送”，无需额外模型 |
| **车牌切片** | **48×160×3，uint8** | **5.81~12.73 KB** | **JPEG q95 编码（真机实测）；切分点产物** |
| 识别输出 | 1×40×6625，float32 | — | 40 时间步 × 6625 类字符，CTC 解码 |

切片的**空间尺寸**由算法固定为 48×160×3，与输入图像无关；JPEG q95 编码后的字节数随切片内容在数 KB 量级浮动。因此**削减倍率是原图体积的函数**（下表切片体积为同一 APK 内 `Bitmap.compress` 的真机实测值）：

| 原图 | 分辨率 | 原图体积 | 切片体积 | 削减 |
|---|---|---|---|---|
| `test_img.jpg` | 1920×1080 | 385.2 KB | 5.81 KB | 98.49%（66×） |
| `2.jpg` | 1140×456 | 2030.8 KB | 12.73 KB | **99.37%（160×）** |

从数据流结构看，端侧完成定位与对齐之后，上传载荷即从 385 KB~2 MB 降至 5.81~12.73 KB；而 Stage 3 的输入恰恰只是这张 KB 级切片而非原图——卸载它既不增加传输量，又移走了流水线中最耗时的深度学习算子；反之若卸载 Detect，则必须上传整张原图，传输代价远大于算力收益。这使**对齐之后**成为最有潜力的候选切分点，其收益的定量评估将在中期报告的划分实验中给出。

由此得到两条对后续划分有约束力的观察：其一，**切分点的本质是“张量体积的悬崖”**——对齐之后载荷从 MB 级跳到 KB 级，这个不连续点由算法结构决定（角点由检测头白送、对齐产出规整小图），与硬件无关，因此找到它比在连续代价曲线上做数值优化更稳健，结论也可迁移到其他视觉流水线；其二，**“是否 AI”不影响切分决策，“单位传输收益”才影响**——对齐虽不是 AI，却是用不到 1% 的算力换来实现切片与原图解耦的枢纽：卸载识别只需上行 5.81 KB 切片，卸载检测必须上行 385 KB~2 MB 原图，**单位传输收益相差两个数量级。**

# 6. 后续工作计划与里程碑

| 序 | 工作内容 | 负责人 / 时间 |
|---|---|---|
| 1 | 四算子解耦：把 `run()` 拆成 Detect / Align / Rec / Cls 四个可独立调用的算子，由外部模块调用原库推理类、对原库零侵入，为端云分工提供运行时接缝 | 队员 2 / 11 月中 |
| 2 | 端侧基线与测试集：已在 Android 真机完成初步基线测量与四算子直接计时；中期前引入 CCPD 公开数据集（MIT 协议）建立分层样本集与 `manifest.csv`，补至 20+ 张自然场景图并建立端云全矩阵 | 队员 2、4 / 11 月中 |
| 3 | 云端联调：队员 3 的 FastAPI 微服务就绪后，接入真实云端服务复跑同一矩阵，替换本地模拟的云侧数据 | 队员 2、3 / 11 月中 |
| 4 | Part C 动态划分：NetworkMonitor + PolicySelector + 运行时热切换，扰动实验验证收敛性 | 组长 / 11 月下 |
| 5 | 多机型真机复测（覆盖不同 SoC 与内存档），并引入能耗代理指标测量 | 队员 2、4 / 12 月上 |
| 6 | 终期报告与 6 分钟演示演练（含演示清单，预演 ≥ 10 次） | 全组 / 12 月中 |

| 里程碑 | 内容 | 时间 |
|---|---|---|
| M1 开题报告 | 选定应用 + 源码模块结构（本报告） | 2026-11-03 提交 / 11-04 汇报 |
| M2 中期报告 | 不同划分方式及其性能的完整对比 | 2026-11-29 |
| M3 终期报告 | 引言 / 正文 / 实验 / 结论 / 参考文献的完整报告 + 源码 | 2026-12-21 |
| M4 现场答辩 | 6 分钟系统演示：划分方案对性能的影响与动态划分实现 | 2026-12-23 |

# 7. 结论

本报告完成了计算划分课题的开题工作：选定 HyperLPR 作为研究对象，完成其源码的模块级剖析——给出模块到文件的分配结构与运行时调用结构，并定位出写死在单一方法内的四阶段流水线的可插拔接缝（算子边界与调用关系已界定，解耦实现按里程碑列入中期）；在此基础上对流水线做数据流建模，量化各节点的中间张量体积，确认对齐之后载荷由原图的 385 KB~2 MB 降至 5.81~12.73 KB（对 2 MB 级原图削减 160 倍），故**对齐之后**是最有潜力的候选切分点——该位置由算法结构决定，不随硬件变化。此外，端侧基线的初步真机测量（1080p 图像单次识别 150.52 ms、进程内存 84.2 MB，四算子逐阶段直接计时）已提前完成，将作为中期划分实验的对照基准。按课程里程碑，各划分方式的设计与性能对比将在 11 月 29 日中期报告中给出，动态自适应划分与系统演示在终期完成。

## 8. 参考文献


1. Cuervo E., Balasubramanian A., Cho D., et al. MAUI: Making Smartphones Last Longer with Code Offload. **Proc. MobiSys**, 2010.
2. Chun B., Ihm S., Maniatis P., et al. CloneCloud: Elastic Execution between Mobile Device and Cloud. **Proc. EuroSys**, 2011.
3. Khan A. U. R., Othman M., Madani S. A., et al. A Survey of Mobile Cloud Computing Application Models. **IEEE Communications Surveys & Tutorials**, 2014.
4. HyperLPR: 开源高性能中文车牌识别框架. GitHub: szad670401/HyperLPR（本团队工作仓库: ya2yo/HyperLPR）.
5. Redmon J., Divvala S., Girshick R., et al. You Only Look Once: Unified, Real-Time Object Detection. **Proc. CVPR**, 2016.
6. Shi B., Bai X., Yao C. An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text. **IEEE TPAMI**, 2017.
7. Flinn J. Balancing Performance, Energy, and Quality in Pervasive Computing. **Proc. ICDCS**, 2002.